

Practical 11

Task 1: Develop a multithreaded counter application using POSIX threads (pthread_create, pthread_join). Demonstrate race conditions by allowing multiple threads to update a shared variable concurrently.

Code:

```
rohita@DESKTOP-1UHU58C: ~  
GNU nano 8.7.1 p11.c *  
#include <stdio.h>  
#include <pthread.h>  
  
int counter = 0;  
  
void *increment(void *arg) {  
    for (int i = 0; i < 100000; i++) {  
        counter++;  
    }  
    return NULL;  
}  
  
int main() {  
    pthread_t t1, t2, t3, t4;  
  
    pthread_create(&t1, NULL, increment, NULL);  
    pthread_create(&t2, NULL, increment, NULL);  
    pthread_create(&t3, NULL, increment, NULL);  
    pthread_create(&t4, NULL, increment, NULL);  
  
    pthread_join(t1, NULL);  
    pthread_join(t2, NULL);  
    pthread_join(t3, NULL);  
    pthread_join(t4, NULL);  
  
    printf("Final counter value: %d\n", counter);  
  
    return 0;  
}
```

Output:

```
rohita@DESKTOP-1UHU58C:~$ nano p11.c
rohita@DESKTOP-1UHU58C:~$ gcc p11.c
rohita@DESKTOP-1UHU58C:~$ ./a.out
Final counter value: 355030
rohita@DESKTOP-1UHU58C:~$ nano p11.c
rohita@DESKTOP-1UHU58C:~$ gcc p11.c
rohita@DESKTOP-1UHU58C:~$ ./a.out
Final counter value: 326840
rohita@DESKTOP-1UHU58C:~$
```

Value is different at each time of execution .

Observation: Multiple threads concurrently updated the shared counter variable, resulting in a race condition. The final value may be less than the expected value because multiple threads access and modify the shared variable simultaneously

Task 2 : Modify the previous program using mutex locks (`pthread_mutex_lock`, `pthread_mutex_unlock`) and compare the outputs with the race-condition version. Analyze the impact of synchronization.

Code:

```

rohita@DESKTOP-1UHU58C: ~
GNU nano 8.7.1                                p11t2.c *
#include <stdio.h>
#include <pthread.h>

#define NUM_THREADS 4
#define ITERATIONS 100000

int counter = 0;
int mutex_counter = 0;

pthread_mutex_t lock;

void *race_increment(void *arg)
{
    for (int i = 0; i < ITERATIONS; i++)
        counter++;

    return NULL;
}

void *mutex_increment(void *arg)
{
    for (int i = 0; i < ITERATIONS; i++) {
        pthread_mutex_lock(&lock);
        mutex_counter++;
        pthread_mutex_unlock(&lock);
    }

    return NULL;
}

int main()
{
    pthread_t threads[NUM_THREADS];

    printf("----- Race Condition Version -----\\n");

    counter = 0;

```

```

    for (int i = 0; i < NUM_THREADS; i++)
        pthread_create(&threads[i], NULL, race_increment, NULL);

    for (int i = 0; i < NUM_THREADS; i++)
        pthread_join(threads[i], NULL);

    printf("Expected counter value: %d\\n", NUM_THREADS * ITERATIONS);
    printf("Actual counter value: %d\\n\\n", counter);

    printf("----- Mutex Version -----\\n");

    mutex_counter = 0;
    pthread_mutex_init(&lock, NULL);

    for (int i = 0; i < NUM_THREADS; i++)
        pthread_create(&threads[i], NULL, mutex_increment, NULL);

    for (int i = 0; i < NUM_THREADS; i++)
        pthread_join(threads[i], NULL);

    printf("Expected counter value: %d\\n", NUM_THREADS * ITERATIONS);
    printf("Actual counter value with mutex: %d\\n", mutex_counter);

    pthread_mutex_destroy(&lock);

    return 0;
}

```

Output:

```
----- Race Condition Version -----  
Expected counter value: 400000  
Actual counter value: 393056  
  
----- Mutex Version -----  
Expected counter value: 400000  
Actual counter value with mutex: 400000  
rohita@DESKTOP-1UHU58C:~$
```

Observation: Mutex synchronization prevents race conditions and produces the correct counter value of 400000, while the unsynchronized version may produce a lower and inconsistent value.